

Programmeren: algoritmen en structuren

De bouwstenen van een programma, de vier datastructuren, de algoritmische technieken, en correctheid en eindigheid.

1 Van probleem naar programma

Een algoritme is een stappenplan dat na eindig veel stappen tot een oplossing komt. Het bestaat los van de taal; pas daarna schrijf je het in **Python**, de programmeertaal van het examen. Je werkt in een online omgeving, zodat je niets op je eigen computer hoeft te installeren.

Je werkt in **vier stappen: het probleem analyseren, een algoritme ontwerpen, het programmeren, en testen en debuggen**. Meteen beginnen typen zonder het probleem te analyseren kost je achteraf meer tijd dan het wint. **Debuggen is fouten opsporen en herstellen**, en je **test ook met randgevallen, omdat fouten zich daar verstoppen**: een lege lijst, 1 element, een 0, een negatief getal.

Geef variabelen een zinvolle naam, zodat je code leesbaar blijft voor jezelf en voor anderen. `aantal_leerlingen` zegt iets, `a` niet. De computer kan het niets schelen, maar een lezer wel, en dat telt mee in de beoordeling. **Commentaar wordt niet mee uitgevoerd**; ze staat er alleen voor wie de code leest.

2 De bouwstenen

Bouwsteen	Wat ze doet	In Python
variabele	een naam waarachter een waarde zit die kan veranderen	na <code>n = 5</code> zit er 5 achter <code>n</code> ; <code>n = 7</code> overschrijft dat
constante	een waarde die tijdens het programma niet verandert	het aantal seconden in een uur, met een naam erbij
conditie	een stuk code alleen laten lopen als iets waar is	<code>if</code> , en <code>else</code> voor wat er anders moet gebeuren
iteratie	een herhaling van hetzelfde stuk code	<code>for</code> over een rij, of <code>while</code> zolang iets geldt
functie	een stuk code een naam geven en hergebruiken	<code>def</code> om ze te schrijven, <code>return</code> voor wat ze teruggeeft

3 De vier datastructuren

Een string is een rij tekens: letters, cijfers en leestekens na elkaar, tussen aanhalingstekens.

Datastructuur	Waarvoor	Kenmerk
<code>list</code>	een geordende rij die je nog wil aanpassen	geordend én aanpasbaar
<code>tuple</code>	een geordende rij die vastligt	je kan hem achteraf niet meer wijzigen
<code>set</code>	enkel de verschillende waarden overhouden	elk element komt er maar één keer in voor
dictionary, in het Nederlands een woordenboek	paren van een sleutel en een waarde	je zoekt iets op met de sleutel

Een list en een tuple zijn allebei geordend; alleen de tuple ligt vast. Een set kan hetzelfde element niet twee keer bevatten: steek je `[1, 2, 2, 3, 3, 3]` erin, dan telt die set nog 3 **elementen**. Dubbels eruit halen is precies waar een set voor dient.

4 Recursie

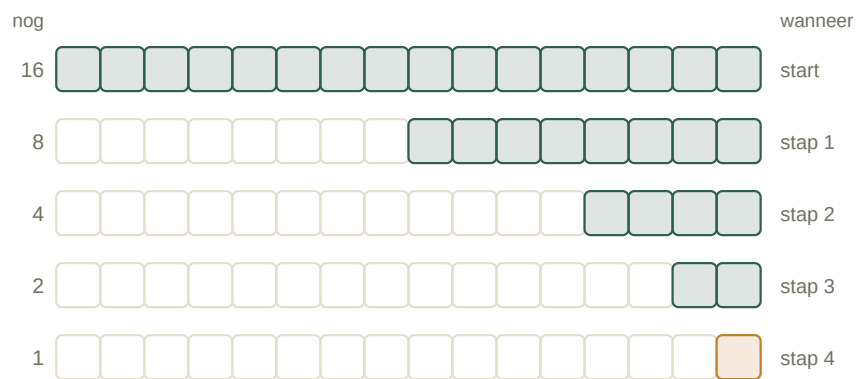
Recursie is een functie die zichzelf oproept, waarbij het probleem telkens een beetje kleiner wordt tot het vanzelf oplosbaar is. De faculteit schrijf je zo: $n! = n \cdot (n - 1)!$, en de rij van Fibonacci zo: $F(n) = F(n - 1) + F(n - 2)$.

Elke recursieve functie heeft een geval nodig waarin ze zichzelf niet meer oproept, een stopgeval: $0! = 1$ bij de faculteit, $F(0) = 0$ en $F(1) = 1$ bij Fibonacci. **Zonder dat blijft ze zichzelf oproepen tot het programma vastloopt**, en geeft Python een foutmelding omdat de oproepen te diep gaan.

5 Verdeel-en-heers

Verdeel-en-heers is de techniek waarbij je **een probleem opsplitst in kleinere deelproblemen en de deeloplossingen daarna samenvoegt**. Sorteren doe je zo: splits de lijst in twee, sorteer elke helft en voeg ze samen.

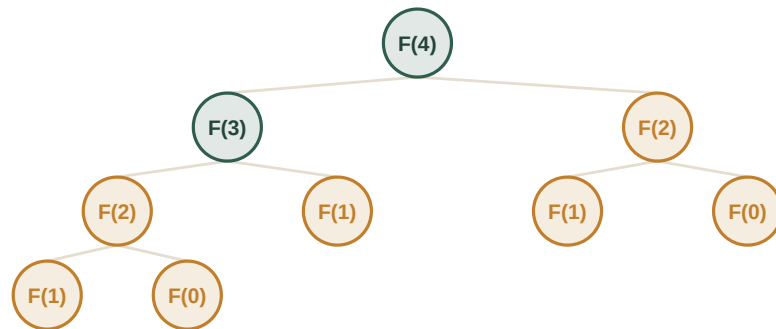
Moet je in een **gesorteerde** rij nagaan of een getal erin staat, dan kijk je in het midden en gooi je **de helft die niet kan kloppen** meteen weg. Van 16 getallen blijven er zo 8 over, dan 4, dan 2, dan 1: 4 stappen, want $2^4 = 16$. Bij 1000 getallen zijn dat er hoogstens 10, want $2^{10} = 1024$; van voor naar achter doorlopen zou er 1000 vragen.



Elke stap valt de helft van de getallen weg. Na vier stappen blijft er nog één over.

6 Dynamisch programmeren

Bij dynamisch programmeren bewaar je tussenresultaten zodat je ze niet twee keer hoeft te berekenen. Reken je $F(n) = F(n - 1) + F(n - 2)$ recursief uit, dan vraag je dezelfde waarden telkens opnieuw.



De oproepen voor $F(4)$. De gekleurde knopen worden meer dan één keer berekend.

Bewaar je elke waarde zodra je ze kent, dan **kost dat meer geheugen maar wint het tijd**: bij Fibonacci scheelt dat een seconde tegenover een eeuwigheid.

7 Correctheid en eindigheid

Een algoritme is correct als het voor elke toegelaten invoer het juiste antwoord geeft, niet alleen voor de gevallen die je toevallig uitprobeerde. **Het is eindig als het na een eindig aantal stappen stopt.**

Je beargumenteert de eindigheid omdat een programma dat blijft lopen niets oplost, ook al klopt elke stap: bij een lus toon je dat de voorwaarde ooit vals wordt, bij recursie dat je het stopgeval altijd bereikt.

8 Twee oplossingen vergelijken

Twee oplossingen voor hetzelfde probleem vergelijk je op snelheid, geheugengebruik en gedrag bij veel gegevens. Twee algoritmen die hetzelfde antwoord geven, zijn daarom nog niet even snel: het ene kan duizend keer trager zijn.

Je schrijft dat kort op met $O(\dots)$: $O(n)$ betekent dat het aantal stappen meegroeit met n , $O(n^2)$ dat het veel sneller stijgt dan n , en $O(\log n)$ dat het nauwelijks stijgt. Bij $n = 1000$ is dat 1000 tegenover 1 000 000 tegenover 10 stappen. Een oplossing die bij 10 getallen vlot werkt, kan bij een miljoen getallen onbruikbaar worden.

 **Weetje.** Zoeken in een gesorteerde rij is $O(\log n)$, precies omdat je elke stap de helft weggooit. Dat is waarom een telefoonboek van een miljoen namen je toch maar twintig keer laat bladeren.

9 Bestanden en bibliotheken

Naast een gewoon tekstbestand lees je ook **CSV**-bestanden in en schrijf je ze weg: bestanden waarin de waarden door komma's of puntkomma's gescheiden staan, zoals een tabel.

Je mag matplotlib, numpy en random gebruiken: een andere bibliotheek importeren mag op het examen niet, tenzij de opdracht er uitdrukkelijk een vermeldt en toelicht.

Matplotlib dient om grafieken te tekenen van je gegevens, **numpy om vlot met grote rijen getallen te rekenen** (veel sneller dan met een lus over een gewone list), en **random om toevalsgetallen te laten genereren**, handig voor een simulatie van dobbelstenen of van een steekproef.

10 Numerieke methoden

Een numerieke methode is een manier om een antwoord te benaderen met rekenstappen. Je krijgt geen exacte formule maar een benadering, die je zo nauwkeurig maakt als je wil.

Een bepaalde integraal benader je met de Riemannsom:

$$\int_a^b f(x) dx \approx \sum_{i=1}^n f(x_i) \cdot \Delta x \quad \text{met} \quad \Delta x = \frac{b-a}{n}$$

Je telt dus **de oppervlakte van heel veel smalle stroken** op. Hoe groter n , hoe smaller elke strook en hoe beter de benadering. De computer doet dat werk in een lus.

Onthoud dit

- Een algoritme is een stappenplan dat na eindig veel stappen tot een oplossing komt.
- Vier stappen: analyseren, een algoritme ontwerpen, programmeren, en testen en debuggen.
- De bouwstenen zijn variabele, constante, conditie, iteratie en functie.
- Een list is aanpasbaar, een tuple ligt vast, een set bevat elk element maar één keer.
- Elke recursieve functie heeft een stopgeval nodig: $0! = 1$, of $F(0) = 0$ en $F(1) = 1$.
- Verdeel-en-heers splitst een probleem in kleinere deelproblemen en voegt de deeloplossingen samen.
- Bij dynamisch programmeren bewaar je tussenresultaten: het kost meer geheugen maar wint tijd.
- Correct is het juiste antwoord voor elke toegelaten invoer; eindig is stoppen na eindig veel stappen.
- Je mag `matplotlib`, `numpy` en `random` gebruiken, en geen andere.